

AI Interview Preparation Guide

Master the questions that matter in AI/ML engineering interviews

6 Topics

18 Questions

Hard / Medium / Easy

Real Practice Tasks

How to Use This Guide

Use STAR-T format: Situation, Task, Approach, Result, and Trade-offs. Always end with trade-offs — it signals seniority.

Speak out loud: Practice answering out loud, not just in your head. The verbal flow matters in real interviews.

Build mini-projects: Index 1000 Wikipedia articles into Pinecone. Build a RAG chatbot with LangChain. 2 hours of building beats 10 hours of reading.

Study Hard questions first: Hard questions (marked H) separate senior from junior candidates. Spend 70% of prep time there.

Know your trade-offs: Every design decision has a trade-off. Interviewers want to see that you understand them, not just the 'right' answer.

Ask clarifying questions: Before answering system design questions, ask about scale, latency requirements, and budget constraints.

1. Indexing & Vector Search

Q1. How would you handle indexing millions of documents for an AI system?

Hard

WHY THEY ASK THIS

Tests knowledge of scalable search and retrieval — core to any production AI system.

KEY POINTS TO COVER

- Use vector databases (Pinecone, Weaviate, pgvector) for semantic similarity search
- Apply chunking: split documents into 256–512 token chunks with overlap
- Use HNSW algorithm for approximate nearest neighbor — $O(\log n)$ lookup
- Hybrid search: combine dense vector search with sparse BM25 for best recall
- Add metadata filters to narrow search space before vector comparison
- Batch indexing pipelines (Kafka + Spark) to process millions asynchronously

Practice: Design a scalable indexing pipeline for 10 million customer support tickets.

Q2. What is the difference between exact nearest neighbor and ANN?

Medium

WHY THEY ASK THIS

ANN vs exact search is a fundamental trade-off — interviewers check if you understand why production systems choose ANN.

KEY POINTS TO COVER

- Exact NN: compares every vector — 100% accurate but $O(n)$ cost, impractical at scale
- ANN: trades small accuracy loss for massive speed gains — HNSW, IVF, LSH are popular
- HNSW builds a multi-layer graph; search time is $O(\log n)$ with ~95%+ recall
- IVF clusters vectors into buckets, only searches nearest clusters
- In practice: ANN gives 95-99% recall at 100x faster speeds than brute force

Practice: Compare HNSW vs IVF for a real-time product recommendation system with 50M vectors.

2. Error Handling in AI Systems

Q1. How do you handle errors in an AI/ML system in production?

Hard

WHY THEY ASK THIS

Production AI errors have unique patterns — model failures, hallucinations, data drift — interviewers want a structured approach.

KEY POINTS TO COVER

- Categorize: model errors, infra errors (timeout, OOM), data errors, API errors
- Circuit breakers: if error rate exceeds threshold, stop traffic and fallback gracefully
- Retry strategies with exponential backoff for transient errors
- Input validation: schema checks, length limits, sanitization before inference
- Output validation: confidence thresholds, toxicity filters on LLM responses
- Dead-letter queues: failed requests go to a queue for reprocessing or human review
- Always have a fallback: rule-based response, cached answer, or 'I don't know'

Practice: Design an error handling strategy for a customer-facing LLM chatbot at 10k req/min.

Q2. How do you communicate an AI system outage to stakeholders?

Easy

WHY THEY ASK THIS

Non-technical communication skill — bridges engineering and business stakeholders.

KEY POINTS TO COVER

- Immediate alert (< 5 min): Slack/email to stakeholders with impact summary
- Update a public/internal status page (e.g. Statuspage.io)
- Impact framing: which features are affected, estimated users impacted, workaround
- Root cause (not blame): 'Our embedding service is unresponsive, search degraded'
- ETA and ownership: name an owner and give a realistic resolution estimate
- Post-incident report within 48 hours: timeline, root cause, preventive actions

Practice: Draft a stakeholder email: your AI recommendation system went down during peak traffic.

3. Monitoring AI Systems

Q1. What metrics would you monitor in a production AI system?

Hard

WHY THEY ASK THIS

AI monitoring goes beyond infra — you need to track model behavior and data health.

KEY POINTS TO COVER

- Infrastructure: latency (p50/p95/p99), throughput, error rate, GPU utilization
- Model quality: accuracy, precision/recall on sampled outputs, hallucination rate
- Data drift: monitor input distribution shifts using KS test, PSI
- Business metrics: task completion rate, user satisfaction, fallback rate
- Cost metrics: tokens per request, cost per query, vector DB query cost
- Tools: Prometheus+Grafana for infra; Langfuse/Arize/W&B; for ML metrics

Practice: Design a monitoring dashboard for a RAG-based support bot — what panels would you include?

Q2. What is data drift and how do you detect it?

Hard

WHY THEY ASK THIS

Data drift is a leading cause of silent model degradation.

KEY POINTS TO COVER

- Input drift: statistical distribution of incoming data shifts from training data
- Concept drift: the relationship between input and output changes over time
- Detect with KS test, Population Stability Index (PSI), Jensen-Shannon divergence
- Automated drift alerts: if PSI > 0.2, trigger model re-evaluation
- Shadow mode: run old and new model in parallel, compare outputs
- Tools: Evidently AI, WhyLogs, Arize Phoenix

Practice: Design a drift detection pipeline for a fraud model processing 1M transactions/day.

4. Building a Customer Support Chatbot

Q1. Walk me through building a customer support chatbot end-to-end.

Hard

WHY THEY ASK THIS

Classic system design question — tests full-stack AI product development understanding.

KEY POINTS TO COVER

- Intent classification: identify what the user wants (refund, tracking, technical issue)
- Knowledge base: index FAQs and docs into a vector DB for RAG
- LLM orchestration: use LangChain to route intent → retrieve context → generate response
- Memory: maintain conversation history, summarize for long chats

- Escalation logic: low confidence or negative sentiment → route to human agent
- Evaluation: measure resolution rate, CSAT, average handle time
- Safety layer: filter harmful inputs, add output guardrails

Practice: Design a chatbot for an e-commerce platform handling returns, tracking, and complaints.

5. Dimensions & Embeddings

Q1. What are embedding dimensions and why do they matter?

Medium

WHY THEY ASK THIS

Dimensions are the core unit of vector representations — tests quality vs cost trade-off understanding.

KEY POINTS TO COVER

- Embedding dimension = number of numerical values in a vector (e.g. 768, 1536)
- Higher dimensions: more semantic nuance, but more memory and compute cost
- Lower dimensions: faster search, less storage, may lose meaning for complex queries
- Common sizes: 384 (lightweight), 768 (BERT), 1536 (OpenAI ada-002), 3072 (large)
- Curse of dimensionality: above ~1000 dims, distance metrics become less meaningful
- Matryoshka embeddings: can truncate to fewer dims without major quality loss

Practice: Choose an embedding model and dimension for a multilingual product search with 5M products.

Q2. What is cosine similarity and when would you use dot product instead?

Medium

WHY THEY ASK THIS

Choosing the wrong similarity metric degrades retrieval quality.

KEY POINTS TO COVER

- Cosine similarity: measures angle between vectors, ignores magnitude
- Dot product: measures both angle and magnitude — faster but sensitive to vector scale
- If embeddings are normalized: cosine and dot product give identical results
- Most embedding models output normalized vectors — use dot product (faster)
- Inner product search is optimized in FAISS/Pinecone — prefer for high throughput

Practice: Which similarity metric would you use for a legal document retrieval system and why?

6. Advanced Topics

Q1. What is RAG and when would you use it over fine-tuning?

Hard

WHY THEY ASK THIS

RAG vs fine-tuning is one of the most common AI architecture questions today.

KEY POINTS TO COVER

- RAG: retrieve relevant documents at query time, inject into LLM context — dynamic, updatable
- Fine-tuning: bake knowledge into model weights — fast inference, static, expensive to update
- Use RAG when: knowledge changes frequently, you need citations, data is proprietary
- Use fine-tuning when: specific tone/format needed, task is narrow, latency is critical
- Hybrid: fine-tune for style + RAG for facts — used by many production systems
- RAG failure modes: retrieval misses, context too long, conflicting retrieved chunks

Practice: A legal firm wants an AI for internal case law Q&A.; RAG, fine-tune, or both?

Q2. What are AI guardrails and how do you implement them?

Medium

WHY THEY ASK THIS

Safety guardrails are now a standard requirement in any AI product.

KEY POINTS TO COVER

- Input guardrails: block harmful prompts, PII detection, topic restriction
- Output guardrails: toxicity filter, fact-checking against retrieved context
- Tools: Guardrails AI, LlamaGuard, NeMo Guardrails
- Prompt injection protection: detect attempts to override system instructions
- Rate limiting and abuse detection: flag users who repeatedly trigger safety filters
- Audit logging: store all inputs/outputs for compliance and debugging

Practice: Design a guardrail system for an AI assistant in a children's educational product.

Q3. Explain the difference between reranking and retrieval in a RAG pipeline.

Hard

WHY THEY ASK THIS

Two-stage retrieval is now standard in production RAG systems.

KEY POINTS TO COVER

- Stage 1 (Retrieval): fast vector search returns top-k candidates (e.g. top 50) — optimized for recall
- Stage 2 (Reranking): cross-encoder scores each candidate vs query — optimized for precision
- Cross-encoders are more accurate but 100x slower — only practical on small candidate sets
- Models: Cohere Rerank, BGE Reranker, ColBERT for fine-grained reranking
- Result: better top-3 results passed to LLM = fewer hallucinations, better answers
- Trade-off: adds ~50-200ms latency — worth it for quality-sensitive applications

Practice: Add a reranking stage to a RAG pipeline for a medical Q&A; system. What model would you use?

Quick Reference Cheat Sheet

Concept	What to Say
Vector DB	Pinecone, Weaviate, pgvector — store and search embeddings
ANN Algorithm	HNSW (graph-based), IVF (cluster-based), both ~95%+ recall
Chunking	256–512 tokens with 20% overlap, sentence-aware boundaries
RAG vs Fine-tune	RAG = updatable facts; Fine-tune = style/format; Hybrid = both
Cosine vs Dot	Both identical on normalized vectors; dot product is faster
Embedding dims	384 (fast), 768 (balanced), 1536 (rich) — benchmark your use case
Monitoring tools	Prometheus+Grafana (infra), Langfuse/Arize (model quality)
Drift detection	KS test, PSI > 0.2 triggers alert; use Evidently AI
Guardrails	LlamaGuard (safety), Guardrails AI (output format), NeMo (rules)
Reranking	Stage 1: fast vector recall; Stage 2: cross-encoder precision
Hallucination fix	RAG + citation check + confidence threshold + human-in-loop
Error handling	Circuit breaker + exponential backoff + dead-letter queue + fallback
Latency optimize	Stream tokens + cache embeddings + smaller model for simple intents

Good luck with your interview!